Base types, primitives
GHC.Base: https://hackage.haskell.org/package/base/docs/GHC-Base.html
GHC.Prim: https://hackage.haskell.org/package/ghc-prim
GHC Unboxed types: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/primitives.html?highlight=primitive
GHC Magic hash: https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/magic_hash.html#magic-hash

GHC.Base

```
-- GHC.Base.Types
data Bool = True | False
data Char = C# Char#
data Int = I# Int#
data Word = W# Word#
data Float = F# Float#
data Double = D# Double#
data Ordering = LT | EQ | GT

newtype IO a = IO (State# RealWorld -> (# State#
RealWorld, a #))
isTrue# :: Int# -> Bool
type Void = (# #)
...
```

GHC.Prim

```
-- GHC.Prim
data Char#
data Int8#
data Int16#
data Int32#
data Int64#
data Int#
data Word8#
data Word16#
data Word32#
data Word64#
data Word#
data Double#
data Float#

data Array#
data MutableArray#
data SmallArray#
...

data Addr#
data State#
data RealWorld#
data ThreadId#
...


andI# :: Int# -> Int# -> Int#
```

```
orI# :: Int# -> Int# -> Int#
xorI# :: Int# -> Int# -> Int#
notI# :: Int# -> Int# -> Int#
(+#) :: Int# -> Int# -> Int#
(-#) :: Int# -> Int# -> Int#
(*#) :: Int# -> Int# -> Int#
(>#) :: Int# -> Int# -> Int#
(>=#) :: Int# -> Int# -> Int#
(==#) :: Int# -> Int# -> Int#
(/=#) :: Int# -> Int# -> Int#
(<#) :: Int# -> Int# -> Int#
(<=#) :: Int# -> Int# -> Int#
...

ord# :: Char# -> Int#
chr# :: Int# -> Char#
int2Word# :: Int# -> Word#
int2Float# :: Int# -> Float#
...

narrowInt8# :: Int# -> Int8#
extendInt8# :: Int8# -> Int#
narrowInt16# :: Int# -> Int16#
...
```

Base types
Base types are defined using **primitive types (unboxed types)** i.e. types for **primitive values** i.e. raw memory values
e.g. Int is defined by primitive type Int#

data Int = I# Int#	Int = base type I# = data constructor for Int Int# = primitive integer type
2 :: Int == I# 2#	2# = primitive value (raw integer in memory)

```
ghci> import GHC.Base
ghci> :set -XMagicHash
ghci> :i Int
type Int :: *
data Int = I# Int#
        -- Defined in 'GHC.Types'
...
ghci> x :: Int; x = 1
ghci> x :: Int; x = I# 1#            -- same
ghci> I# 1# == 1
True
```

```
---- Basics
--
-- Primitive types
{-# MagicHash #-}
```

```
import GHC.Prim
x :: Int#
x = 1#
x :: Word#
x = 1##
x :: Float#
x = 1.1#
x :: Double#
x = 1.1##

x :: Char#
x = 'a'#
x :: Addr#
x = "abc"#

-- convert (GHC.Prim 0.6)
narrowInt8 :: Int8# -> Int#
extendInt8 :: Int# -> Int8#
x :: Int8
x = narrrowInt8# 1#
x :: Word8
x = narrowWord8# 1#
...


-- operators
(+#) :: Int# -> Int# -> Int#
(-#) :: Int# -> Int# -> Int#
(*#) :: Int# -> Int# -> Int#
(>#) :: Int# -> Int# -> Int#
(#) :: Int# -> Int# -> Int#
(#) :: Int# -> Int# -> Int#
(#) :: Int# -> Int# -> Int#
(#) :: Int# -> Int# -> Int#
(#) :: Int# -> Int# -> Int#
(#) :: Int# -> Int# -> Int#
1# +# 2#
```